

Methods of String

=====

```
1.public char charAt(int index)
2.public String concat(String str)
3.public boolean equals(Object o)
4.public boolean equalsIgnoreCase(String s)
5.public String substring(int begin)
6.public String substring(int begin,int end)
7.public int length()
8.public String replace(char old,char new)
9.public String toLowerCase()
10.public String toUpperCase()
11.public String trim()
12.public int indexOf(char ch)
13.public int lastIndexOf(char ch)
```

```
3.public boolean equals(Object o)
```

It is used for Content Comparison, In String class equals() method is Overriden to check the content of the object

```
4.public boolean equalsIgnoreCase(String s)
```

It is used for Content Comparison without comparing the case.

eg#1.

```
public class Test {
    public static void main(String[] args) {
        String s ="java";
        System.out.println(s.equals("JAVA")); //false
        System.out.println(s.equalsIgnoreCase("JAVA")); //true
    }
}
```

Assignment

=====

```
credentials(gmail)
username:nitin@ineuron.ai(not case sensitive)
password :***** (case sensitive)
```

```
5.public String substring(int begin)
```

It gives the String from the begin index to end of the String.

```
String s="Ineeuron";
System.out.print(s.substring(2)); //searching from 2 to end of the string
```

```
6.public String substring(int begin,int end)
```

It gives the String from the begin index to end-1 of the String.

```
String s="Ineeuron";
System.out.print(s.substring(2,6)); //searching from 2 to 5 will happen
```

eg#1.

```
public class Test {
    public static void main(String[] args) {
        String s ="sachinINDMI";
        System.out.println(s.length());

        System.out.println();

        System.out.println(s.substring(9));
    }
}
```

```

        System.out.println(s.substring(0,8));
        System.out.println(s.substring(0,9));
    }
}

8. public String replace(char old,char new)
    String s="ababab";
    System.out.print(s.replace('a','b')); //bbbbbb

eg#1.
public class Test {
    public static void main(String[] args) {
        String name ="sbchin";
        System.out.println(name.replace('b','a')); //sachin

        String data = "ababab";
        System.out.println(data.replace('a','b')); //bbbbbb
    }
}

9. public String toLowerCase()
10. public String toUpperCase()

eg:
public class Test {
    public static void main(String[] args) {
        String name ="sAchIn";//mixedCase
        System.out.println(name.toUpperCase()); //SACHIN
        System.out.println(name.toLowerCase()); //sachin
    }
}

11. public String toString();
    Note: When ever we print any reference,by default JVM will call
    toString() on the reference
        eg: System.out.println(name);
            System.out.pritnln(name.toString());

eg#1.
//userdefined class
class Student{
    String name = "sachin";
    int id = 10;
}
public class Test {
    public static void main(String[] args) {

        Student std = new Student();
        System.out.println(std);//Student@HexadecimalValue
        System.out.println(std.toString());////Student@HexadecimalValue

        System.out.println();

        String name =new String("dhoni");
        System.out.println(name);//dhoni
        System.out.println(name.toString());//dhoni
    }
}

```

```
}
```

```
11.public String trim()
```

To remove the blank spaces present at the beginning and end of string but not the blank spaces present at the middle of the String.

eg#1.

```
public class Test {  
    public static void main(String[] args) {  
        String name = "Sachin IND";  
        System.out.println(name.length()); //10  
        System.out.println(name.trim()); //Sachin IND  
  
        System.out.println();  
  
        String state = " Karnataka ";  
        System.out.println(state.length()); //13  
        System.out.println(state.trim()); //Karnataka  
    }  
}
```

```
12.public int indexOf(char ch)
```

It returns the index of 1st occurrence of the specified character if the specified character is not available then it returns -1.

```
String s="sachinramesh";  
System.out.print(s.indexOf('a')); //1  
System.out.print(s.indexOf('z')); //-1
```

```
13.public int lastIndexOf(char ch)
```

It returns the index of last occurrence of the specified character if the specified character is not available then it returns -1.

```
String s="sachinramesh";  
System.out.print(s.lastIndexOf('a')); //7  
System.out.print(s.lastIndexOf('z')); //-1
```

eg#1.

```
public class Test {  
    public static void main(String[] args) {  
        String name = "hyderAbbasbengaluru";  
  
        System.out.println();  
  
        System.out.println(name.indexOf('A')); //5  
        System.out.println(name.indexOf('a')); //8  
  
        System.out.println();  
  
        System.out.println(name.indexOf('b')); //6  
        System.out.println(name.lastIndexOf('b')); //10  
  
        System.out.println();  
        System.out.println(name.lastIndexOf('Z')); //-1
```

```
    }  
}
```

Predict the output

=====

```
Q>  
String s1="sachin";                // s1,s3 -> sachin (scp)  
String s2=s1.toUpperCase(); // s2->SACHIN(heap area)  
String s3=s1.toLowerCase();  
System.out.print(s1==s2);//false  
System.out.print(s1==s3);//true
```

```
Q>  
String s1="sachin";                // s1,s2-> sachin (SCP)  
String s2=s1.toString();  
System.out.print(s1==s2);//true
```

```
Q>  
String s1=new String("sachin");  
String s2=s1.toString();  
String s3=s1.toUpperCase();  
String s4=s1.toLowerCase();  
String s5=s1.toUpperCase();  
String s6=s1.toLowerCase();  
System.out.print(s1==s6);//true  
System.out.print(s3==s5);//false
```

final vs Immutability

=====

=> final is a modifier applicable for variables, where as immutability is applicable only for Objects.

=> If reference variable is declared as final, it means we cannot perform reAssignment for the reference variable,

it doesnot mean we cannot perform any change in that object.

=> By declaring a reference variable as final, we wont get immutability nature.

=> final and Immutability is different concept.

```
eg:: final StringBuilder sb=new StringBuilder("sachin");  
      sb.append("tendulkar");  
      System.out.println(sb);  
      sb=new StringBuilder("dhoni"); //CE::Cannot assign a value to final  
variable
```

Note::

```
final variable(valid),  
final object(invalid),  
immutable variable(invalid)  
immutable object(valid)
```

StringBuilder, StringBuffer are by default mutable.

All Wrapper classes(Byte, Short, Long, Integer, Float, Double, Boolean, Character) are by Default Immutable.

Mutable -> can be changed

Immutable => can't be changed

StringBuffer

=====

1. If the content will change frequently then it is not recommended to go for String object because for every new change a new Object will be created.
2. To handle this type of requirement, we have StringBuffer/StringBuilder concept

Constructors of StringBuffer

=====

1. `StringBuffer sb=new StringBuffer();`
creates a empty StringBuffer object with default initial capacity of "16".
Once StringBuffer reaches its maximum capacity a new StringBuffer Object will be created
$$\text{new capacity} = (\text{current capacity} + 1) * 2;$$

```
eg1::StringBuffer sb = new StringBuffer();
      System.out.println(sb.capacity());//16
      sb.append("abcdefghijklmnp");
      System.out.println(sb.capacity());//16
      sb.append('q');
      System.out.println(sb.capacity());//34
```

2. `StringBuffer sb=new StringBuffer(initialCapacity);`
It creates an Empty String with the specified initial capacity.

```
eg1::StringBuffer sb = new StringBuffer(19);
      System.out.println(sb.capacity());//19
```

3. `StringBuffer sb=new StringBuffer(String s);`
It creates a StringBuffer object for the given String with the capacity = `s.length() + 16;`

```
eg1::StringBuffer sb = new StringBuffer("sachin");
      System.out.println(sb.capacity());//22
```

Important methods of StringBuffer/StringBuilder

=====

- a. `public int length()`
- b. `public int capacity()`
- c. `public char charAt(int index)`
- d. `public void setCharAt(int index, char ch)`

=====

- e. `public StringBuffer append(String s)`
- f. `public StringBuffer append(int i)`
- g. `public StringBuffer append(long l)`
- h. `public StringBuffer append(boolean b)`
- i. `public StringBuffer append(double d)`
- j. `public StringBuffer append(float f)`
- k. `public StringBuffer append(int index, Object o)`

=====

- l. `public StringBuffer insert(int index, String s)`
- m. `public StringBuffer insert(int index, int i)`
- n. `public StringBuffer insert(int index, long l)`
- o. `public StringBuffer insert(int index, double d)`
- p. `public StringBuffer insert(int index, boolean b)`
- q. `public StringBuffer insert(int index, float f)`
- r. `public StringBuffer insert(int index, Object o)`

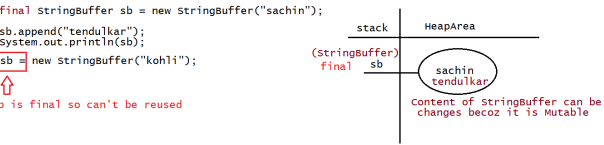
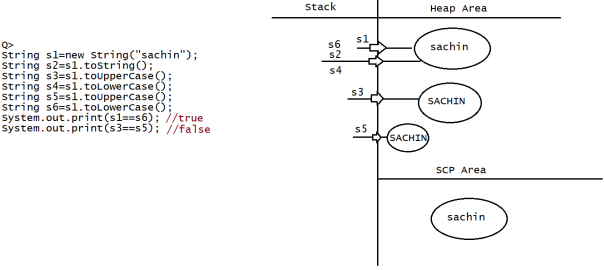
```
=====
public StringBuffer delete(int begin,int end)
public StringBuffer deleteCharAt(int index)
public StringBuffer reverse()
public void setLength(int Length)
public void trimToSize()
public void ensureCapacity(int capacity)
```

eg::

```
StringBuilder sb = new StringBuilder("sachinrameshtendulkar");
System.out.println(sb.length());//21
System.out.println(sb.capacity());//21 + 16 = 37
System.out.println(sb.charAt(20));//'r'
System.out.println(sb.charAt(100));//StringIndexOutOfBoundsException
```

eg::

```
StringBuffer sb1 =new StringBuffer("kohlianushka");
sb1.setCharAt(5, 'A');
System.out.println(sb1);//kohliAnushka
```

```

Q> int data[] = {2010,2013,2014,2015,2014};
    int key = 2014;
    int count=0;
    for(int e:data){
        if(e!=key){
            continue;
            count++;
        }
    }
    System.out.println(count+" found");

```

What is the result?

- A. Compilation fails
- B. 0 found
- C. 1 found
- D. 3 found

Answer: A (unreachable code becoz of continue)

```

Q>
String[] arr = {"A","B","C","D"};
for(int i =0;i< arr.length;i++){
    System.out.print(arr[i]+" ");
    if(arr[i].equals("C"))
        continue;
    System.out.println("Work done");
    break;
}

```

- A. A B C Work done
- B. A B C D Work done
- C. A Work done
- D. Compilation fails

```

arr[0] = "A", arr[1]= "B", arr[2] ="C", arr[3]="D"
i =0, arr.length=4
0<4(true)
A Work done

```

Answer: C

```

Q>
String[] str = new String[2];
int idx = 0;
for(String s: str){
    str[idx].concat(" element " + idx);
    idx++;
}
for(idx =0;idx<str.length;idx++){
    System.out.print(str[idx]);
}

```

- A. element 0
element 1
- B. null element 0
null element 1
- C. null
null
- D. NullPointerException is thrown at runtime

Answer: D

Q>

```
String[][] arr = {"A","B","C"}, {"D","E"};
for(int i =0;i< arr.length;i++){
    for(int j=0;j<arr[i].length;j++){
        System.out.print(arr[i][j]+" ");
        if(arr[i][j].equals("B"))
            break;
    }
    continue;
}
```

- A. A B C
- B. A B C D E
- C. A B D E
- D. Compilation fails

Answer: C

Q>

```
public class Test {
    public static void main(String[] args) {
        StringBuffer sb =new StringBuffer("java");
        String s = "java";
        if (sb.toString().equals(s.toString()))
            System.out.println("Match -1");
        else if(sb.equals(s))
            System.out.println("Match -2");
        else
            System.out.println("No Match");
    }
}
```

- A. Match -1
- B. Match -2
- C. No Match
- D. Null Pointer excpetion at runtime

Answer: A

Q> int[] a =new int[]? What is the array size ?

- A. 0
- B. 4
- C. 1
- D. Can't find it results in Compiletime error
- E. exception at the runtime

Answer: D

Q> int[] a =new int[0]; will the code compile?

- A. yes
- B. no

Answer: A

Q> int[] a=new int[-5];? What it the size of the array?

- A. 5
- B. -5
- C. 0
- D. 1

- F. Compilation error as size is negative
- G. NegativeArraySizeException is occurred
- H. ArrayIndexOutOfBoundsException is occurred

Answer: G

ArrayIndexOutOfBoundsException => Array is already created and if we go out of index while reading or writing to array we get.

NegativeArraySizeException is occurred => During creation of an Array it would result.

Q> Consider below code:

```
public class Test {
    public static void main(String[] args) {
        System.out.println("ONE");
    }
    public static void main(Integer[] args) {
        System.out.println("TWO");
    }
    public static void main(byte [] args) {
        System.out.println("THREE");
    }
}
```

What will be the result if Test class is executed by below command?

java Test 10

- A. TWO
- B. ONE
- C. THREE
- D. Compilation Error

Answer: B

Q>

```
public class Test {
    public static void main(String[] args) {
        if ((isItSmall(3)) || (isItSmall(7))) { //line-3
            System.out.println("Result is true");
        }
        if ((isItSmall(6)) || (isItSmall(9))) { //line-6
            System.out.println("Result is true");
        }
    }
    public static boolean isItSmall(int i) {
        if (i < 5) {
            System.out.println("i < 5");
            return true;
        } else {
            System.out.println("i >= 5");
            return false;
        }
    }
}
```

What is the result?

- A. Compilation Error at line 3
- B. Compilation Error at line 6
- C. i<5
Result is true
- D. i<5
Result is true

i>=5
Result is true

E. i<5
Result is true
i>=5
i>=5
Result is true

F. i<5
Result is true
i>=5
i>=5

Answer: F

Q>
class Feline {
 public static void main(String[] args) {
 Long x = 42L;
 Long y = 44L;
 System.out.print(" " + 7 + 2 + " "); // 72
 System.out.print(foo() + x + 5 + " "); //foo425
 System.out.println(x + y + foo()); //86foo
 }
 static String foo() { return "foo"; }
}

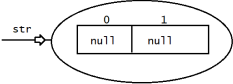
What is the result?

- A. 9 foo47 86foo
- B. 9 foo47 4244foo
- C. 9 foo425 86foo
- D. 9 foo425 4244foo
- E. 72 foo47 86foo
- F. 72 foo47 4244foo
- G. 72 foo425 86foo
- H. 72 foo425 4244foo

Answer: G

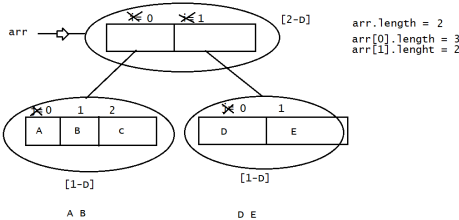

```
String[] str = new String[2];
int idx = 0;

for(String s: str){
    str[idx].concat(" element " + idx);
    idx++;
}
for(idx =0;idx<str.length;idx++){
    System.out.print(str[idx]);
}
```



```
s = null.concat(" ")
NullPointerException
```

```
String[] arr = {{ "A","B","C"}, {"D","E"} };
for(int i =0;i< arr.length;i++){
    for(int j=0;j<arr[i].length;j++){
        System.out.print(arr[i][j]+" " );
        if(arr[i][j].equals( "B"))
            break;
    }
    continue;
}
```



```
arr.length = 2
arr[0].length = 3
arr[1].length = 2
```

```
StringBuffer sb =new StringBuffer("java");
String s = "java";

if (sb.toString().equals(s.toString()))
    System.out.println("Match -1");
else if(sb.equals(s))
    System.out.println("Match -2");
else
    System.out.println("No Match");
```

